WEEK 1 · LESSON 5 OF 24 · HANDOUT

Packaging with Docker and Configuration

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=Jq68axbKIbg — open the lesson online to watch inside the course.

🎓 **Go deeper:** EuroPython 2020: Best practices for production-ready Docker packaging — https://www.youtube.com/watch?v=H0UPw-ILiz4

🎓 **Go deeper:** PyParis: Deep Learning Model Deployment with ONNX — https://www.youtube.com/watch?v=cswXUNtUfuI

Your goal in this lesson: Package a model service so it runs the same everywhere using Docker, keep settings and secrets in environment variables, and produce a ready-to-deploy folder.

"It works on my computer"

Your service needs a certain Python version, certain libraries, the model file and some settings. On another computer one of these is often different — and the service breaks. **Docker** packages the code, libraries and model into an **image**; a running copy of an image is a **container**. The same image runs the same way on your laptop, a school server or a cloud droplet.

```
import numpy as np
import pandas as pd

def make_applicants(n=3000, seed=7, drift=0.0, concept=0.0):
    # Made-up SACCO loan applications. drift > 0: incomes, loans and late payments change.
    # concept > 0: the link to repayment changes (salaried jobs protect less, late payments
hurt more).
    rng = np.random.default_rng(seed)
    gender = rng.choice(["female", "male"], n)
    area = rng.choice(["urban", "rural"], n, p=[0.6, 0.4])
    rural = area == "rural"
    income = (rng.lognormal(12.4, 0.5, n) * np.where(rural, 0.7, 1.0) * (1 - drift *
0.25)).round(-3)
    loan = (rng.uniform(50_000, 600_000, n) * (1 + drift)).round(-3)
    years = rng.integers(0, 13, n)
    late = rng.poisson(0.7 + drift, n)
    salaried = (rng.random(n) < np.where(rural, 0.3, 0.6)).astype(int)
    momo = np.clip(rng.normal(np.where(rural, 18, 45), 10), 0, None).round().astype(int)
    score = 3.0 - 1.6 * (loan / income) + 0.12 * years - 0.7 * late + 0.6 * salaried + 0.01 *
momo
    score = score - concept * (1.5 * salaried + 0.6 * late - 0.04 * (momo - 30))
    repaid = (rng.random(n) < 1 / (1 + np.exp(-score))).astype(int)
    officer = score - 0.9 * rural - 0.6 * (gender == "female") + rng.normal(0, 0.6, n)
    return pd.DataFrame({
        "applicant_id": np.arange(1, n + 1), "age": rng.integers(20, 63, n), "gender": gender,
"area": area,
        "income_rwf": income, "loan_rwf": loan, "years_member": years, "late_payments": late,
        "salaried": salaried, "momo_tx_month": momo,
        "approved_before": (officer > 1.2).astype(int), "repaid": repaid})

applicants = make_applicants()
print(applicants.shape, "| repaid:", round(applicants["repaid"].mean(), 3))
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

FEATURES = ["income_rwf", "loan_rwf", "years_member", "late_payments", "salaried",
"momo_tx_month"]
TARGET = "repaid"
train, test = train_test_split(applicants, test_size=0.25, random_state=0,
stratify=applicants[TARGET])
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(train[FEATURES], train[TARGET])
proba = model.predict_proba(test[FEATURES])[:, 1]
print(f"accuracy {accuracy_score(test[TARGET], proba >= 0.5):.3f} | AUC
{roc_auc_score(test[TARGET], proba):.3f}")
```

Settings come from the environment

Never write paths, thresholds or secrets inside the code. Read them from **environment variables**, with safe defaults:

```
import os

def load_settings():
    return {
        "model_path": os.environ.get("MODEL_PATH", "model/loan_model.joblib"),
        "threshold": float(os.environ.get("APPROVE_THRESHOLD", "0.5")),
        "model_version": os.environ.get("MODEL_VERSION", "1.0.0"),
        "api_key": os.environ.get("API_KEY"),            # a secret: no default, never in the
code
    }

print(load_settings())
os.environ["APPROVE_THRESHOLD"] = "0.6"
print("after changing the environment:", load_settings()["threshold"])
del os.environ["APPROVE_THRESHOLD"]
```

The same image can now run with different settings (test or production) without changing code. Secrets such as API keys or database passwords are given to the container when it starts — they are **never** stored in the code, the image or Git.

Build the service folder

This cell writes every file the service needs into a folder `loan_service/`:

```
import pathlib
import importlib.metadata as md
import joblib

svc = pathlib.Path("loan_service")
(svc / "model").mkdir(parents=True, exist_ok=True)
joblib.dump(model, svc / "model" / "loan_model.joblib")

(svc / "main.py").write_text('''import os

import joblib
import pandas as pd
from fastapi import FastAPI
from pydantic import BaseModel, Field

MODEL_PATH = os.environ.get("MODEL_PATH", "model/loan_model.joblib")
THRESHOLD = float(os.environ.get("APPROVE_THRESHOLD", "0.5"))
VERSION = os.environ.get("MODEL_VERSION", "1.0.0")
FEATURES = ["income_rwf", "loan_rwf", "years_member", "late_payments", "salaried",
"momo_tx_month"]

app = FastAPI(title="SACCO loan scoring", version=VERSION)
model = joblib.load(MODEL_PATH)


class Applicant(BaseModel):
    income_rwf: float = Field(..., ge=10_000, le=20_000_000)
    loan_rwf: float = Field(..., ge=10_000, le=5_000_000)
    years_member: int = Field(..., ge=0, le=60)
    late_payments: int = Field(..., ge=0, le=50)
    salaried: int = Field(..., ge=0, le=1)
    momo_tx_month: int = Field(..., ge=0, le=1000)


@app.get("/health")
def health():
    return {"status": "ok", "model_version": VERSION}


@app.post("/predict")
def predict(applicant: Applicant):
    p = float(model.predict_proba(pd.DataFrame([applicant.model_dump()])[FEATURES])[0, 1])
    return {"repay_probability": round(p, 3), "approve": p >= THRESHOLD, "model_version":
VERSION}
''')

libs = ["fastapi", "uvicorn", "scikit-learn", "pandas", "joblib"]
pins = []
for lib in libs:
    try:
        pins.append(f"{lib}=={md.version(lib)}")
    except md.PackageNotFoundError:
        pins.append(lib)                                    # not installed here: leave unpinned
(svc / "requirements.txt").write_text("\n".join(pins) + "\n")

(svc / "Dockerfile").write_text('''FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN useradd --create-home appuser
USER appuser
```

```
ENV MODEL_PATH=model/loan_model.joblib
EXPOSE 8000
HEALTHCHECK CMD python -c "import urllib.request;
urllib.request.urlopen('http://localhost:8000/health')"
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
''')
(svc / ".dockerignore").write_text("__pycache__/\n*.ipynb\n.env\ndata/\n")

for path in sorted(svc.rglob("*")):
    if path.is_file():
        print(f"{str(path):40s} {path.stat().st_size:>8,} bytes")
```

What the Dockerfile does, line by line:

1. Start from a small official Python image.
2. Install the pinned libraries first (Docker caches this step, so rebuilds are fast).
3. Copy the code and model.
4. Create and switch to a normal user — never run services as root.
5. Declare the port and a **health check** that calls `/health`.
6. Start the API with uvicorn.

`.dockerignore` keeps notebooks, secrets (`.env`) and raw data out of the image.

Check the service before you ship it

```
import importlib
import sys
from fastapi.testclient import TestClient

sys.path.insert(0, str(svc.resolve()))
cwd = os.getcwd(); os.chdir(svc)
service = importlib.import_module("main")
client = TestClient(service.app)
print(client.get("/health").json())
print(client.post("/predict", json={"income_rwf": 400_000, "loan_rwf": 150_000, "years_member":
6,
                                    "late_payments": 0, "salaried": 1, "momo_tx_month":
50}).json())
os.chdir(cwd)

import shutil
print("download this file:", shutil.make_archive("loan_service", "zip", svc))
```

In Colab, download `loan_service.zip` from the Files panel.

Build and run with Docker

On a computer or server with Docker installed:

```
cd loan_service
docker build -t loan-service:1.0.0 .
docker run -d --name loan-service -p 8000:8000 \
  -e MODEL_VERSION=1.0.0 -e APPROVE_THRESHOLD=0.5 -e API_KEY="$LOAN_API_KEY" \
  loan-service:1.0.0
curl http://localhost:8000/health
```

Tag images with the version (`1.0.0`), never only `latest`, so you can always go back to a known version.

Key terms

- **Docker image** — a package of code, libraries and files.
- **Container** — a running copy of an image.
- **Dockerfile** — the recipe for building an image.
- **Environment variable** — a setting given to a program from outside.
- **Secret** — a password or key; never in code, images or Git.
- **Health check** — a regular test that the service is alive.

Remember this: Docker packages the service so it runs the same everywhere; settings and secrets come from environment variables, and every image has a version tag and a health check.

Name: _____ Date: _____

Activity

Task 1

Start the service code with `APPROVE_THRESHOLD=0.7` and compare decisions for three applicants.

Task 2

Add `MODEL_VERSION` to the `/predict` answer and change it through the environment.

Task 3

If you have Docker, build and run the image, then open `http://localhost:8000/docs`.

Task 4

Explain to a classmate why `API_KEY` has no default value in `load_settings`.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What is a Docker container?

   A. A database
   B. A running copy of a Docker image
   C. A type of model
   D. A Python library

2. Where should an API key for the service come from?

   A. A public Git repository
   B. An environment variable or secret store given at start-up
   C. Written in main.py
   D. The Docker image

3. Why install requirements before copying the code in the Dockerfile?

   A. Docker caches that step, so rebuilds after code changes are fast
   B. It makes the image bigger
   C. It is required
   D. To run as root

4. Why create a normal user in the Dockerfile?

   A. Python needs it
   B. Services should not run as root, for security
   C. To save space
   D. For speed

5. What does .dockerignore do?

   A. Keeps files such as notebooks, .env and raw data out of the image
   B. Ignores errors
   C. Speeds up the model
   D. Deletes Docker

6. Why tag images with a version like 1.0.0?

   A. It looks nice
   B. So you can always run or return to a known version
   C. To hide the model
   D. Docker requires numbers

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.